

Задача:

Реализация утилиты `bldd` (обратный `ldd`), которая показывает все исполняемые файлы, использующие указанные файлы разделяемых библиотек.

Требования:

- Возможность настройки директории сканирования
- Поддержка архитектур `x86`, `x86_64`, `armv7`, `aarch64`
- Наличие команды `--help` с примерами

Выполнение работы и результаты:

1) Создание файла для тестов на C:

```

namkitu@namkitu-IdeaPad:~/labs/lab1/tests$ cat lab1_test.c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>      // Потребуется libm
#include <pthread.h>    // Потребуется libpthread
#include <unistd.h>

// Функция для выполнения в потоке
void* calculate_sqrt(void* arg) {
    double* num = (double*)arg;
    printf("[Thread] Вычисляем корень из %.2f...\n", *num);
    sleep(1);
    double result = sqrt(*num);
    printf("[Thread] Результат: %.4f\n", result);
    return NULL;
}

int main(int argc, char* argv[]) {
    if (argc < 2) {
        printf("Использование: %s <число>\n", argv[0]);
        return 1;
    }

    double input = atof(argv[1]);
    pthread_t thread;

    printf("[Main] Запуск потока для математических вычислений...\n");

    // Создание потока
    if (pthread_create(&thread, NULL, calculate_sqrt, &input) != 0) {
        perror("Ошибка создания потока");
        return 1;
    }

    // Ожидание завершения потока
    pthread_join(thread, NULL);

    printf("[Main] Программа завершена.\n");
    return 0;
}

```

2) Кросс-компиляция под разные архитектуры с помощью скрипта:

```

namkitu@namkitu-IdeaPad:~/labs/lab1/tests$ ./compile_tests.sh
namkitu@namkitu-IdeaPad:~/labs/lab1/tests$ ls
compile_tests.sh  lab1_arm_test  lab1_x86_64_test
lab1_arm64_test  lab1_test.c    lab1_x86_test

```

3) Скрипт compile_tests.sh для кросс-компиляции:

```
namkitu@namkitu-IdeaPad:~/labs/lab1/tests$ cat compile_tests.sh
#!/bin/bash

# x86_64
gcc -o lab1_x86_64_test lab1_test.c -pthread -lm -ldl

# x86
gcc -m32 -o lab1_x86_test lab1_test.c -pthread -lm -ldl

# ARM64
aarch64-linux-gnu-gcc-13 -o lab1_arm64_test lab1_test.c -pthread -lm -ldl

# ARM
arm-linux-gnueabihf-gcc-13 -o lab1_arm_test lab1_test.c -pthread -lm -ldl
```

4) Вызов команды --help:

```
namkitu@namkitu-IdeaPad:~/labs/lab1$ ./blld.py --help
usage: blld.py [-h] [-d DIRECTORY] [-o OUTPUT]

blld (Reverse LDD)

options:
  -h, --help            show this help message and exit
  -d DIRECTORY, --directory DIRECTORY
                        Directory to scan (by default: ./)
  -o OUTPUT, --output OUTPUT
                        Path to save report (txt)

Usage examples:
  python3 blld.py -d /usr/bin
  python3 blld.py --directory /home/user/my_project --output report.txt
```

5) Вызов генерации отчета по подготовленным тестам:

```
namkitu@namkitu-IdeaPad:~/labs/lab1$ ./blld.py -d ./tests -o report.txt
[INFO] Scanning directory: ./tests
[INFO] Scan complete. Found 4 ELF executables.
[INFO] Report saved to report.txt
```

6) Содержимое файла report.txt:

```

namkitu@namkitu-IdeaPad:~/labs/lab1$ cat report.txt
Report on dynamic used libraries
Generated by bldd-python-wrapper
=====

----- arm64 -----
libm.so.6 (1 execs)
    -> ./tests/lab1_arm64_test
libc.so.6 (1 execs)
    -> ./tests/lab1_arm64_test
ld-linux-aarch64.so.1 (1 execs)
    -> ./tests/lab1_arm64_test

----- armv7 -----
libm.so.6 (1 execs)
    -> ./tests/lab1_arm_test
libc.so.6 (1 execs)
    -> ./tests/lab1_arm_test
ld-linux-armhf.so.3 (1 execs)
    -> ./tests/lab1_arm_test

----- x86 -----
libm.so.6 (1 execs)
    -> ./tests/lab1_x86_test
libc.so.6 (1 execs)
    -> ./tests/lab1_x86_test

----- x86-64 -----
libm.so.6 (1 execs)
    -> ./tests/lab1_x86_64_test
libc.so.6 (1 execs)
    -> ./tests/lab1_x86_64_test

```

Программа была реализована на python3

Исходный код:

```
#!/usr/bin/env python3
```

```
import os
```

```
import re
```

```
import argparse
```

```
import subprocess
```

```
from collections import defaultdict
```

```
from concurrent.futures import ThreadPoolExecutor
```

```
ARCH_REGEX = re.compile(r"Machine:\s+(.+)")
```

```
NEEDED_REGEX = re.compile(r"\(NEEDED\) \s+Shared library: \[(.+) \]")
```

```
ARCH_MAP = {
```

```
    "Advanced Micro Devices X86-64": "x86-64",
```

```

    "Intel 80386": "x86",
    "AArch64": "arm64",
    "ARM": "armv7"
}
ELF_MAGIC = b'\x7fELF'

def check_elf_magic(path):
    try:
        with open(path, 'rb') as f:
            return f.read(4) == ELF_MAGIC
    except Exception:
        return False

def analyze_file(path):
    try:
        arch_proc = subprocess.run(['readelf', '-h', path], capture_output=True,
text=True)
        if arch_proc.returncode != 0:
            return None

        arch_match = ARCH_REGEX.search(arch_proc.stdout)
        arch = arch_match.group(1).strip() if arch_match else "Unknown"
        arch = ARCH_MAP.get(arch, arch)

        libs_proc = subprocess.run(['readelf', '-d', path], capture_output=True,
text=True)
        libs = NEEDED_REGEX.findall(libs_proc.stdout)

        if not libs:
            return None

        return {
            'path': path,
            'arch': arch,
            'libs': libs
        }
    except Exception as e:
        return None

def generate_report(results, output_path=None):
    data = defaultdict(lambda: defaultdict(list))

    for item in results:
        for lib in item['libs']:
            data[item['arch']][lib].append(item['path'])

```

```

lines = [
    "Report on dynamic used libraries",
    "Generated by bldd-python-wrapper",
    "=" * 40,
    ""
]

for arch in sorted(data.keys()):
    lines.append(f"----- {arch} -----")

    libs = data[arch]
    sorted_libs = sorted(libs.items(), key=lambda x: len(x[1]), reverse=True)

    for lib, paths in sorted_libs:
        lines.append(f"{lib} ({len(paths)} execs)")
        for p in paths:
            lines.append(f"\t-> {p}")
        lines.append("")

report_text = "\n".join(lines)

if output_path:
    with open(output_path, 'w', encoding='utf-8') as f:
        f.write(report_text)
    print(f"[INFO] Report saved to {output_path}")
else:
    print(report_text)

def main():
    parser = argparse.ArgumentParser(
        description="bldd (Reverse LDD)",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Usage examples:
python3 bldd.py -d /usr/bin
python3 bldd.py --directory /home/user/my_project --output report.txt
        """
    )
    parser.add_argument("-d", "--directory", default=".", help="Directory to scan (by default: .)")
    parser.add_argument("-o", "--output", help="Path to save report (txt)")

    args = parser.parse_args()

```

```
if not os.path.isdir(args.directory):
    print(f"Error: Directory '{args.directory}' does not exist.")
    return

print(f"[INFO] Scanning directory: {args.directory}")

all_files = []
for root, _, files in os.walk(args.directory):
    for name in files:
        full_path = os.path.join(root, name)
        if check_elf_magic(full_path):
            all_files.append(full_path)

with ThreadPoolExecutor() as executor:
    results = list(filter(None, executor.map(analyze_file, all_files)))

print(f"[INFO] Scan complete. Found {len(results)} ELF executables.")
generate_report(results, args.output)

if __name__ == "__main__":
    main()
```